

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación

Contenido

Introducción	04
Ecosistema base: Python, pip, PyPI y venv	04
¿Qué es venv (entorno virtual)?	05
¿Qué es Flask? Fundamentos	05
¿Cómo usamos Flask? Primeros Pasos	06
¿Qué es una ruta? Rutas y Decoradores	07
Estructura mínima y ciclo request/response	08
Plantillas y Jinja2	09
¿Cómo usamos las plantillas?	09
Archivos estáticos (dentro de "static/")	11
Construcción de URLs con url_for	12
Podemos utilizar URLs con parámetros, ¿Cómo funciona?	13
Formularios, métodos HTTP y request	14
Ingresar información con un formulario	15
Manejo de ERRORES y Buenas Prácticas	17
Escalando Blueprints y App Factory	17
¿Qué es un Blueprint?	18
¿Qué es una App Factory?	19
Recomendaciones al usar Blueprints y App Factory	20
Mostrar información de una lista en una plantilla	20
Bibliografía	24

Clase 7



iTe damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- Ecosistema de un proyecto usando Flask.
- Instalación de Flask y primeros pasos.
- Plantillas con Jinja2.
- Manejo de errores y formularios.
- Modularidad usando blueprints y app factory.
- Información dinámica.

Introducción

En la clase anterior repasamos modelado de casos de uso, análisis estructural y orientado a objetos, y la trazabilidad. Todo esto forma parte del análisis dentro del ciclo de vida del software.

Ahora, en esta clase, vamos a dar un paso más: llevar nuestras ideas a la práctica con código real. El framework elegido es Flask, un entorno de desarrollo web en Python que nos permitirá empezar a construir aplicaciones funcionales.

El propósito de esta clase es pasar del análisis (casos de uso, DFD/DER, trazabilidad) a un primer prototipo web funcional en Python con Flask, entendiendo qué hace cada pieza y por qué.

Ecosistema base: Python, pip, PyPI y venv

¿Qué es pip?

pip es el gestor oficial de paquetes de Python. Se conecta a PyPI (Python Package Index) —el repositorio público— para descargar e instalar librerías de terceros (por ejemplo, Flask).

- Resolución de dependencias: cuando instalás un paquete, pip descarga también los paquetes que éste necesita (dependencias).
- Formatos de distribución: instala wheels (.whl, binarios precompilados) o sdist (código fuente).
- Versionado: soporta especificadores como flask==3.0.0, flask>=2.3,<3.0.

Reproducibilidad: con `pip freeze > requirements.txt` capturás versiones exactas para que otra persona pueda recrear el entorno con `pip install -r requirements.txt`.

Comandos útiles:

- `python -m pip install paquete` (forma recomendada)
- `pip list`, `pip show flask`, `pip uninstall paquete`
- `pip install .` para instalar un paquete local

Analogía: pip es tu App Store para Python: buscás, instalás, actualizás y removés “apps” (librerías) para tu proyecto.

¿Qué es venv (entorno virtual)?

venv crea un entorno aislado por proyecto. Las librerías instaladas ahí no afectan a tu sistema ni a otros proyectos, es útil si queremos instalar librerías que solo utilizaremos en un único proyecto, nos ahorra espacio en memoria y para organizarnos si queremos trabajar con distintas versiones, nos viene bien.

Comandos base:

- Crear: `python -m venv .venv`
- Activar (Windows): `.\.venv\Scripts\activate`
- Activar (macOS/Linux): `source .venv/bin/activate`
- Ventajas: aislamiento, reproducibilidad, evitar conflictos de versiones.

¿Qué es Flask? Fundamentos

Flask es un microframework web para Python que implementa el patrón WSGI (interfaz estándar entre servidores web y apps Python). Su filosofía es minimalista: te da lo esencial (rutas, plantillas, request/response), y vos agregás lo que necesitás (BD, auth, formularios con CSRF, etc.).

WSGI (v1.0.1 / PEP 3333): define cómo el servidor web (p. ej., Werkzeug / Gunicorn) llama a tu app Python para procesar una solicitud HTTP.

Componentes clave:

- Flask (app y rutas).
- Werkzeug (servidor WSGI y utilidades).
- Jinja2 (motor de plantillas HTML).

¿Por qué Flask?:

Curva de aprendizaje suave, ideal para prototipos, APIs y microservicios; gran comunidad.

¿Cómo usamos Flask? Primeros Pasos

Para comenzar, primero lo primero instalamos flask en cmd o terminal donde tenemos nuestros proyectos de Python:

"pip install flask"

Luego, en un archivo llamado "app.py" Tenemos el siguiente código:

```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def index():
7      return "Hola a todos"
8
9  if __name__ == '__main__':
10     app.run(debug=True)
```

Expliquemos estas líneas de código...

¿Qué es una ruta? Rutas y Decoradores

Cada vez que queramos utilizar flask en nuestro proyecto de python, lo primero es importar la librería, lo hacemos con "import Flask" o "from flask import Flask" ambas hacen lo mismo.

app = Flask(__name__):

- Crea una instancia de la aplicación Flask.
- Flask usa __name__ para saber dónde buscar archivos como templates, static, etc.
- Sin esto, Flask no podría encontrar tus archivos HTML, CSS o imágenes correctamente.

@app.route('/')

En Flask, **@app.route** es un decorador que se utiliza para vincular una URL específica a una función.

En otras palabras, define qué función de tu aplicación Flask se debe ejecutar cuando un usuario accede a una URL determinada en tu aplicación web.

Lo primero que tenemos que hacer es activar el comando flask, lo siguiente que podemos hacer es ver que retorna la ruta principal del proyecto.

```
9  if __name__ == '__main__':  
10  app.run(debug=True)
```

Esto hace que podamos levantar el servidor local y podamos verlo en nuestro navegador.

Si ejecutamos este código, ¡tenemos nuestra primera web en python funcionando!

Estructura mínima y ciclo request/response

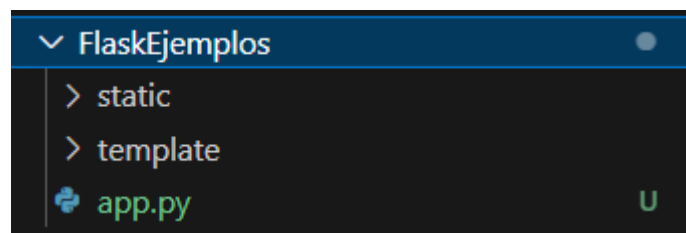
La estructura mínima y general que sigue un proyecto en flask es la siguiente:

nombre_proyecto/

app.py

/templates

/static



((Dentro de la carpeta del proyecto tenemos el archivo de la aplicación, y tendremos directorios "templates" y "static" cada uno de ellos cumple un rol específico))

El ciclo básico es en esencia "request/response" (simplificado):

1. El usuario ingresa una URL.
2. Flask busca si tenés una ruta para esa URL.
3. Ejecuta la función asociada (vista).
4. Devuelve la respuesta (texto, HTML o JSON).

```
@app.route('/saludo')
def saludo():
    return "Hola mundo desde otra ruta"
```

Si escribimos en el navegador "http://127.0.0.1:5000/saludo" podremos ver el mensaje.

En definitiva, una ruta vincula una URL con una función. Como ya vimos, se definen con `@app.route('...')`.

Cómo construimos rutas:

```
16  # Ruta de contacto
17  @app.route('/contacto')
18  def contacto():
19      return "Página de contacto"
20
```

Probemos acceder a `/contacto` en el navegador.

Buenas prácticas:

- Nombrar la función con intención (home, about, list_users).
- Usar parámetros de ruta (`/users/<int:id>`).
- Devolver tipos claros (string/HTML/JSON).

Plantillas y Jinja2

Flask no solo devuelve texto plano, también puede renderizar HTML desde la carpeta templates. Para inyectar datos usamos Jinja2.

Jinja2 es el motor de plantillas de Flask. Permite inyectar variables en HTML y usar estructuras de control (for/if) del lado del servidor.

- **Variables:** `{{ nombre }}`
- **Control:** `{% for x in xs %}...{% endfor %}`, `{% if cond %}...{% endif %}`
- **Autoescape:** por defecto Jinja escapa contenido para prevenir XSS.

¿Cómo usamos las plantillas?

En nuestro archivo app.py

```

28 #Mostrando plantillas
29 from flask import render_template
30
31 @app.route('/')
32 def home():
33     return render_template('index.html')

```

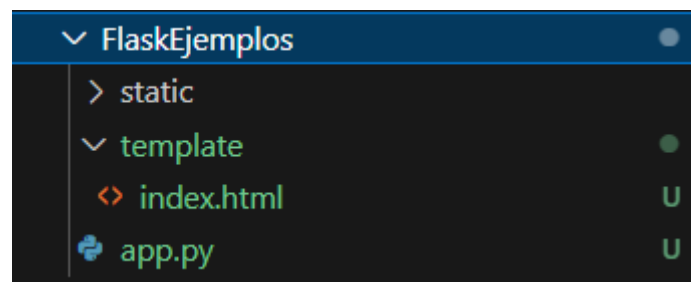
La plantilla "index.html":

```

1 <!DOCTYPE html>
2 <html>
3     <head><title>Página inicial</title></head>
4     <body>
5         <h1>Bienvenidos a Flask</h1>
6     </body>
7 </html>
8

```

Estructura de cómo quedaría nuestro proyecto:



Si lo probamos, ahora la ruta raíz devuelve un HTML completo.

Jinja2 son moldes (plantillas) para hornear páginas con "rellenos" (datos).

¿Cómo podemos pasarle datos a nuestras plantillas?

El método "render_template" puede recibir por parámetro lo que nosotros le indiquemos, esto para que después en la plantilla se utilice, ejemplo:

```

35 #Pasando datos a las plantillas
36 @app.route('/usuario')
37 def usuario():
38     return render_template('usuario.html', nombre="Da Vinci")

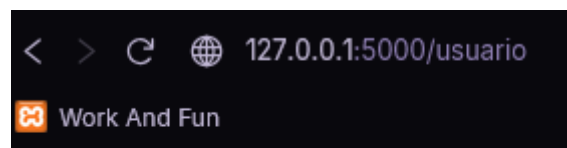
```

```

<title>Soy Usuario.html</title>
</head>
<body>
  <h2>Hola, {{ nombre }}!</h2>
</body>
</html>

```

Lo que veríamos:

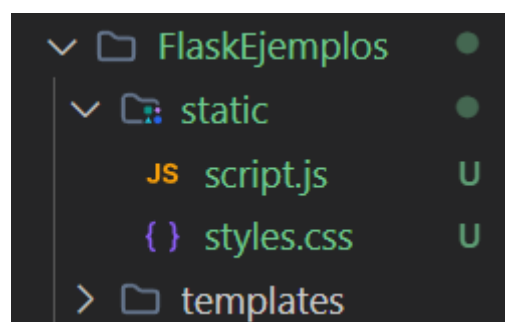


Hola, Da Vinci!

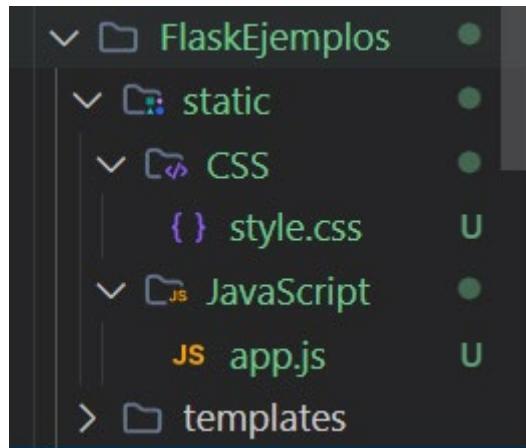
Hasta ahora estamos viendo cosas sencillas sobre Front-End en definitiva, solo que integrado a Python, ¿Podremos entonces integrar hojas de estilos y javascript? De hecho, sí. Veamos:

Archivos estáticos (dentro de "static/")

Cómo vimos antes, en nuestra estructura del proyecto tendremos un directorio "static", dentro de este es donde nosotros podemos alojar los archivos .css y .js, es decir, los estilos y scripts que vamos a utilizar en nuestra web van en esa carpeta "static/..".



Ejemplo alojando archivos en /static



Aplicando buenas prácticas, en /static separamos en carpetas “/CSS” y “/JavaScript”

*¿Cómo podemos usarlo? En nuestro archivo .html en el que vayamos a consumir lo que está dentro de “static”, usamos “**url_for('static', filename=nombre_carpeta/nombre_archivo.extension')**”.*

Ejemplo aplicado en nuestro archivo index.html:

```
<head>
  <link rel="stylesheet" href="{{ url_for('static', filename='CSS/style.css') }}">
</head>
<body>
  <h1>Ejemplo con estilos</h1>
  <script src="{{ url_for('static', filename='JavaScript/app.js') }}"></script>
</body>
```

Por qué url_for: genera URLs correctas incluso si cambiás prefijos o configuraciones.

Construcción de URLs con url_for

En Python usando flask, “url_for” genera rutas de forma dinámica a partir del nombre de la función, no de un string fijo.

En nuestro **app.py**:

```
10 @app.route('/about')
11 def about():
12     return "Acerca de nosotros"
13
```

En nuestro archivo **index.html**:

```
<h1>Creando url con "url_for":</h1>
<a href="{{ url_for('about') }}">Ir a Acerca de</a>
```

*El link lleva a la ruta /about. Si en otro momento cambiamos la ruta a /**acerca**, url_for se actualiza solo, ya que usa el nombre de la función "about" y no del string '/acerca', esto hace que evite "Error 404".*

Podemos utilizar URLs con parámetros, ¿Cómo funciona?

Es muy común encontrarnos que muchas veces las URLs necesitan parámetros dinámicos.

Ejemplo: Mostrar el perfil de un usuario según su ID.

En nuestro "app.py":

```
#Parametros en URLs
@app.route('/usuario/<int:id>')
def perfil(id):
    return f"Perfil del usuario {id}"
```

Y lo usamos en nuestro **index.html** de la siguiente manera:

```
<h1>Pasando parámetros en URLs:</h1>
<a href="{{ url_for('perfil', id=10) }}">Ver perfil</a>
```

A esto podemos sacarle mucho provecho, cómo por ejemplo pasándole información desde Python, trabajando con sesiones, bucles, etc.

Formularios, métodos HTTP y request

Los formularios permiten que el usuario envíe datos. En Flask se capturan con "request".

- **request:** Es el objeto que representa la solicitud (datos de formulario, query strings, cabeceras).
- **GET:** Manda los datos directamente en la URL. Esto puede ser peligroso si lo usamos incorrectamente, normalmente se usa GET para búsquedas, filtros, navegación, etc.
- **POST:** Manda los datos en el cuerpo de la solicitud (más seguro). Normalmente usamos POST para login de usuario, comentarios, formularios con datos sensibles, etc.

Para utilizar esto en Python, importamos "request"

```
Projects-In-Python > FlaskEjemplos >  app.py > ...  
1   from flask import Flask, request  
2
```

¿Y cómo lo utilizamos?, en nuestro app.py, ejemplo usando POST:

```
#Formulario en Flask  
@app.route('/formulario')  
def formulario():  
    return render_template('form.html')  
  
@app.route('/procesar', methods=['POST'])  
def procesar():  
    nombre = request.form.get('nombre')  
    return f"Hola {nombre}, tus datos fueron recibidos."
```

Y luego en nuestro archivo "form.html" de la siguiente manera:

```
<form action="{{ url_for('procesar') }}" method="POST">
  <input type="text" name="nombre" placeholder="Escribí tu nombre">
  <button type="submit">Enviar</button>
</form>
```

¡Prueben eso y vean los resultados!

Ingresar información con un formulario

¿Y si queremos que el usuario sea el que ingrese los datos y nosotros procesar esos datos? Veamos cómo hacerlo.

Ya sabemos que funciona con los métodos GET y POST, y la manera en cómo podemos capturar los datos es usando **"request.form.get('campo')"**

En nuestro archivo app.py:

```
#Capturando Datos de un Formulario

from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def formulario():
    return render_template('formulario.html')

@app.route('/submit', methods=['POST'])
def submit():
    nombre = request.form.get('nombre')
    email = request.form.get('email')
    return render_template('resultado.html', nombre=nombre, email=email)
```

Creamos nuestro archivo formulario.html:


```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Formulario</title>
5  </head>
6  <body>
7  |   <h1>Formulario de registro</h1>
8  |   <form action="{{ url_for('submit') }}" method="POST">
9  |       <label>Nombre:</label>
10 |       <input type="text" name="nombre" required><br><br>
11 |
12 |       <label>Email:</label>
13 |       <input type="email" name="email" required><br><br>
14 |
15 |       <button type="submit">Enviar</button>
16 |   </form>
17 </body>
18 </html>

```

Y para mostrar los datos, creamos el archivo "resultado.html":

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Datos Recibidos</title>
5  </head>
6  <body>
7  |   <h1>Datos Recibidos</h1>
8  |   <p>Nombre: {{ nombre }}</p>
9  |   <p>Email: {{ email }}</p>
10 </body>
11 </html>

```

¡Podemos jugar con esto! Prueben pedirle al usuario más datos y mostrarlos en pantalla

Manejo de ERRORES y Buenas Prácticas

Con Flask, tenemos la posibilidad de personalizar las páginas de error, como la de "404 no encontrado".

¿Cómo?

Usando decoradores "@app.errorhandler(404)" / @app.errorhandler(500) con plantillas dedicadas.

En nuestro archivo app.py:

```
#Manejo de Error 404
@app.errorhandler(404)
def pagina_no_encontrada(e):
    return render_template('404.html'), 404
```

Creamos el archivo HTML (404.html) personalizado:

```
<h1>Error 404</h1>
<p>La página que buscás no existe.</p>
<a href="{{ url_for('inicio') }}">Volver al inicio</a>
```

Esto se puede sacar provecho si queremos poner animaciones o cosas de ese estilo para hacerlo más divertido.

Escalando Blueprints y App Factory

Cuando hacemos proyectos simples, usamos un único archivo app.py.

Pero en proyectos más grandes:

1. El archivo se vuelve enorme y difícil de mantener.
2. Varios programadores trabajan en distintas partes del sistema (usuarios, productos, reportes).
3. Necesitamos ordenar el código en módulos independientes.

Ahí entran en juego:

- Blueprints → permiten crear módulos de rutas “independientes”.
- App Factory → patrón que centraliza la creación de la aplicación Flask, en vez de crearla “fija” en un solo archivo.

Imaginá tu app como una ciudad.

- Cada Blueprint sería un “barrio” (usuarios, productos, etc.), que tiene sus propias calles (rutas).
- La App Factory es la “constructora” que arma la ciudad completa y conecta los barrios entre sí.

¿Qué es un Blueprint?

Un Blueprint es como una mini aplicación dentro de tu aplicación principal.

- Tiene sus propias rutas.
- Puede tener sus propias plantillas.
- Puede tener sus propios archivos estáticos.
- Al final, se registra dentro de la aplicación principal.

¿Cómo lo usamos? De la siguiente manera:

```
#Usando blueprint en flask
from flask import Blueprint

usuarios_bp = Blueprint('usuarios', __name__)

@usuarios_bp.route('/')
def lista():
    return "Listado de usuarios"
```

Acá creamos un Blueprint llamado usuarios_bp.

'usuarios': nombre interno (identificador).

`__name__`: ayuda a Flask a ubicar recursos relativos (plantillas, estáticos).

Definimos una ruta (/) que devuelve un string.

Ojo: este Blueprint todavía no funciona solo. Hay que conectarlo a la aplicación principal con `app.register_blueprint(...)`.

¿Qué es una App Factory?

Hasta el momento para usar nuestra app hacíamos **"app = Flask(__name__)"**

Eso crea la app de forma fija.

Con una App Factory, en cambio, creamos una función que devuelve la aplicación ya configurada.

```
1  #Usando App Factory
2
3  from flask import Flask
4
5  def create_app():
6      app = Flask(__name__)
7
8      from .usuarios.routes import usuarios_bp
9      app.register_blueprint(usuarios_bp, url_prefix='/usuarios')
10
11     from .productos.routes import productos_bp
12     app.register_blueprint(productos_bp, url_prefix='/productos')
13
14     return app
```

- La función `create_app()` construye la aplicación.
- `app.register_blueprint(usuarios_bp, url_prefix='/usuarios')` conecta el módulo de usuarios y le dice que todas sus rutas empiezan con `/usuarios`.
- Ahora nuestra app no existe hasta que la fabriquemos con `create_app()`.

Dentro de la función `"create_app()"` tenemos dos módulos, uno para usuarios y otro para productos y al final lo que devuelve la función es la aplicación configurada (en este caso, con los dos módulos).

Recomendaciones al usar Blueprints y App Factory

Una estructura del proyecto recomendada para no tener un desorden, es la siguiente:

```
project/
|
├─ wsgi.py                # Punto de entrada
├─ /app
|   ├─ __init__.py        # App Factory
|   ├─ /usuarios
|   |   ├─ __init__.py
|   |   └─ routes.py      # Rutas del módulo usuarios
|   └─ /templates         # Plantillas compartidas
```

Ventajas de esta arquitectura:

- **Escalabilidad:** podés tener un módulo para usuarios, otro para productos, otro para reportes, etc.
- **Colaboración:** cada integrante del equipo trabaja en su propio Blueprint sin romper a los demás.
- **Configuración flexible:** podés tener `create_app(testing=True)` para usar configuraciones distintas en desarrollo, testing y producción.
- **Buenas prácticas industriales:** es la forma que se usa en proyectos Flask profesionales.

Mostrar información de una lista en una plantilla

En Flask podemos pasar datos desde Python hacia HTML.

Esto nos permite mostrar información que no está fija en la página, sino que se genera dinámicamente.

Para esto usamos Jinja2, que es el motor de plantillas integrado en Flask.

Claves de Jinja2:

- **{{ variable }}** → muestra el valor de una variable.
- **{% for item in lista %}** → recorre listas o colecciones.
- **{% if condición %}** → evalúa condiciones.

Ejemplo práctico de cómo podemos utilizarlo:

en nuestro **app.py**

```
1  from flask import Flask, render_template
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def home():
7      usuarios = ['Da Vinci', 'Ghami', 'Elias', 'Facu', 'Sistema']
8      return render_template('usuarios.html', usuarios=usuarios)
9
10 if __name__ == '__main__':
11     app.run(debug=True)
```

Y ahora en “**usuarios.html**”:

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Lista de Usuarios</title>
5  </head>
6  <body>
7  |   <h1>Usuarios registrados</h1>
8  |   <ul>
9  |       {% for usuario in usuarios %}
10 |         <li>{{ usuario }}</li>
11 |       {% endfor %}
12 |   </ul>
13 </body>
14 </html>

```

Explicación de lo que hicimos:

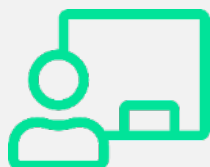
1. En Python creamos una lista: ['Da Vinci', 'Ghami', 'Elias', 'Facu', 'Sistema'].
2. Esa lista se pasa a la plantilla con `render_template('usuarios.html', usuarios=usuarios)`.
3. En HTML usamos un bucle for de Jinja2 para recorrerla.
4. Cada elemento de la lista se imprime con `{{ usuario }}`.

¡Pruebenlo! Lo que debería pasar es que al abrir la página veríamos la lista de nombres en viñetas ().



Hemos llegado así al final de esta clase en la que vimos:

- Ecosistema de un proyecto usando Flask.
- Instalación de Flask y primeros pasos.
- Plantillas con Jinja2.
- Manejo de errores y formularios.
- Modularidad usando blueprints y app factory.
- Información dinámica.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.
¡Hasta la próxima clase!

Bibliografía

Grinberg, M. (2018). Flask web development: Developing web applications with Python (2.^a ed.). O'Reilly Media.

<https://www.oreilly.com/library/view/flask-web-development/9781491991725/>

Pallets Projects. (2024). Flask Documentation. Flask.

<https://flask.palletsprojects.com/>

Python Packaging Authority. (2024). pip documentation. PyPA.

<https://pip.pypa.io/>

Python Software Foundation. (2024). venv — Creation of virtual environments. PSF. <https://docs.python.org/3/library/venv.html>

Python Packaging Authority. (2024). Python Package Index (PyPI).

PyPI. <https://pypi.org/>